

UNITÀ
00

LEZIONE
01

Python, oggetti e prime istruzioni

Ripasso dei fondamenti del linguaggio

Prof. Francesco Gobbi

Anno scolastico 2026/2027

I.I.S.S. Galileo Galilei – Ostiglia (MN)

Che cos'è Python?

Python è un **linguaggio di programmazione ad alto livello**, di uso generale e multiparadigma: supporta diversi stili di programmazione.

Caratteristiche

- ▶ sintassi relativamente semplice;
- ▶ codice molto leggibile;
- ▶ tipizzazione dinamica;
- ▶ gestione automatica della memoria;
- ▶ vasta libreria standard.

Paradigmi

- ▶ programmazione procedurale;
- ▶ programmazione a oggetti;
- ▶ programmazione funzionale;

Python permette al programmatore di concentrarsi soprattutto sulla **logica del problema**, nascondendo molti dettagli a basso livello.

Le peculiarità di Python

Leggibile

sintassi essenziale
e codice chiaro

Dinamico

le variabili possono
riferirsi a oggetti
di tipi differenti

A oggetti

i dati sono
rappresentati
come oggetti

Python viene utilizzato in numerosi ambiti:

Web – **Data Science** – **AI** – **Automazione** – **Ricerca**

Dal codice all'esecuzione



L'**interprete Python** esegue le istruzioni che compongono il programma.

Se incontra un problema durante l'analisi o l'esecuzione, produce un **messaggio di errore**.

Dove possiamo programmare in Python?

Python può essere utilizzato attraverso ambienti di sviluppo differenti.

IDE

VS Code

PyCharm

IDLE

Il codice viene normalmente
salvato in file `.py`.

Notebook

Jupyter

JupyterLab

Codice organizzato in celle
eseguibili.

Cloud

Google Colab

Kaggle

L'ambiente funziona
direttamente online.

Nel nostro corso utilizzeremo principalmente **Google Colab**, che permette di lavorare con notebook Jupyter direttamente nel browser.

Che cos'è Jupyter Notebook?

Un **Jupyter Notebook** è un documento interattivo composto da celle.

CODICE

istruzioni Python

OUTPUT

risultati

TESTO

spiegazioni

Una cella di codice può essere **eseguita individualmente**.

Il risultato viene mostrato direttamente al di sotto della cella stessa.

Le celle condividono la memoria della sessione: conta l'ordine in cui vengono eseguite. Dopo una modifica, riesegui anche le celle che usano quel risultato.

Google Colab: il nostro workspace

Google Colab mette a disposizione un **workspace Jupyter completamente online**.

Possiamo

- ▶ scrivere codice Python;
- ▶ eseguire singole celle;
- ▶ osservare immediatamente l'output;
- ▶ inserire spiegazioni testuali;
- ▶ salvare il notebook online.

Vantaggi

- ▶ nessuna installazione locale;
- ▶ ambiente accessibile dal browser;
- ▶ notebook facilmente condivisibili;
- ▶ utile anche per Data Science e AI.

Il notebook diventa il nostro **spazio di lavoro**: codice, risultati e spiegazioni convivono nello stesso documento.

Google Colab e Kaggle

Google Colab

- ▶ notebook Jupyter online;
- ▶ semplice da utilizzare;
- ▶ adatto alla didattica;
- ▶ integrato nell'ecosistema Google;
- ▶ esecuzione Python nel cloud.

Kaggle

- ▶ notebook online;
- ▶ orientato alla Data Science;
- ▶ integrazione con dataset;
- ▶ risorse di calcolo cloud;
- ▶ diffuso nel Machine Learning.

Entrambi permettono di programmare in Python direttamente online. Durante il corso utilizzeremo principalmente **Google Colab**.

Le prime righe di codice

CODICE PYTHON

```
a = 2
b = 3
somma = a + b

print("Ciao, classe!")
print("Ripassiamo Python")
print(somma)
```

OUTPUT

```
Ciao, classe!
Ripassiamo Python
5
```

Python esegue normalmente le istruzioni **dall'alto verso il basso**.

L'espressione `a + b` viene valutata: il risultato è un oggetto, al quale viene associata l'etichetta `somma`.

Il paradigma orientato agli oggetti

Un **paradigma di programmazione** è un modo di organizzare la soluzione di un problema attraverso il codice.

Nella **programmazione orientata agli oggetti**, o **OOP**, il programma lavora con entità che riuniscono:

- ▶ **stato**: i dati che descrivono la loro situazione;
- ▶ **comportamento**: le operazioni che possono svolgere.

Per rappresentare un'automobile possiamo descrivere il suo **colore** e la sua **velocità**, insieme all'operazione **accelerare**. La classe stabilisce il modello comune; ogni oggetto rappresenta una singola automobile.

Che cos'è una classe?

Una **classe** è una definizione scritta nel programma che introduce un **tipo di oggetto**: descrive come inizializzare le sue istanze e quali operazioni esse mettono a disposizione.

Possiamo interpretarla come un **modello comune** che specifica:

- ▶ quali dati vogliamo associare a ogni istanza, attraverso gli **attributi**;
- ▶ come impostare il suo **stato iniziale**;
- ▶ quali operazioni definire, attraverso i **metodi**.

Con la classe `Automobile` descriviamo come rappresentare le automobili nel programma. I valori concreti, come "blu", caratterizzano le singole istanze.

La classe Automobile: il modello comune

CLASSE Automobile

Dati previsti per ogni istanza

colore: una stringa, scelta quando si crea l'oggetto.

velocita: un numero, inizialmente uguale a 0.

Operazione disponibile

`accelera(incremento)`: aumenta la velocità dell'istanza.

Definire la classe stabilisce questo modello. Per ottenere una singola automobile dobbiamo poi **creare un'istanza**.

Che cos'è un oggetto?

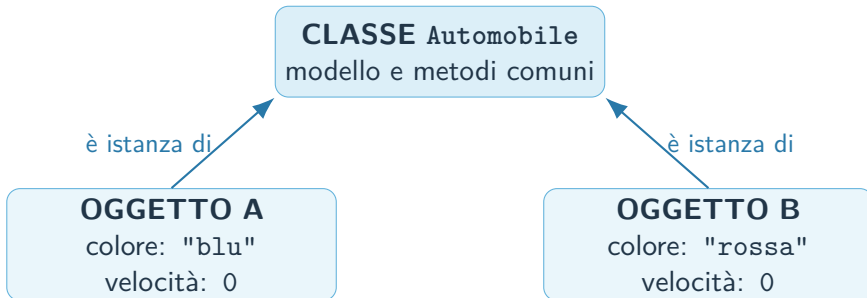
Un **oggetto** è un'entità concreta gestita dal programma. È un'**istanza di una classe**: appartiene a quel tipo e può usare le operazioni che la classe rende disponibili.

Ogni oggetto possiede:

- ▶ una **classe**, che ne determina il tipo;
- ▶ una propria **identità**, che lo distingue dagli altri oggetti;
- ▶ un **valore o stato**, cioè i dati che lo caratterizzano in quel momento.

Per esempio: un'istanza di Automobile, con colore "blu" e velocità 0, rappresenta **una specifica automobile**.

Una classe, due oggetti distinti



La classe è la stessa, ma ciascuna istanza ha una propria identità e i propri attributi di istanza. Anche due automobili entrambe blu possono essere oggetti distinti.

Gli attributi descrivono lo stato dell'oggetto

Un **attributo di istanza** è un dato associato a una specifica istanza. L'insieme dei valori degli attributi ne descrive lo **stato**.

Attributo	Oggetto A	Oggetto B
colore	"blu"	"rossa"
velocita	0	30

Se la variabile `auto` fa riferimento all'oggetto A, `auto.colore` permette di accedere al suo attributo `colore`. Il punto collega l'oggetto all'attributo che vogliamo consultare.

I metodi definiscono il comportamento

Un **metodo di istanza** è una funzione definita nella classe che, quando viene chiamata tramite un'istanza, riceve un riferimento a quell'istanza.

```
auto.accelera(20)
```

- ▶ `auto` fa riferimento all'oggetto su cui operare.
- ▶ `accelera` indica il metodo da chiamare.
- ▶ `20` è l'argomento che specifica l'aumento di velocità.

Nel nostro esempio il metodo modifica l'attributo `velocita` di quella specifica automobile. Altri metodi possono leggere lo stato o restituire un risultato, senza modificare l'oggetto.

La definizione della classe in Python

CODICE PYTHON

```
class Automobile:
    def __init__(self, colore):
        self.colore = colore
        self.velocita = 0

    def accelera(self, incremento):
        self.velocita += incremento
```

`class Automobile:` introduce la classe.

`def` definisce i metodi.

L'indentazione raggruppa le istruzioni che appartengono a ciascun blocco.

`self.colore` e `self.velocita` sono attributi dell'istanza.

Questo codice definisce il modello `Automobile`. Non crea ancora alcuna automobile. La sola definizione della classe non produce output.

Il ruolo di `self` e di `__init__`

- ▶ `self` è il parametro che fa riferimento all'istanza su cui sta operando il metodo. Python lo passa automaticamente quando chiamiamo il metodo tramite l'oggetto.
- ▶ `__init__` è un metodo speciale: inizializza un'istanza appena creata. Non è il metodo che crea materialmente l'oggetto.

In `self.colore = colore`, la parte destra `colore` è il parametro che riferisce il valore ricevuto. La parte sinistra `self.colore` è l'attributo che viene impostato su quella specifica istanza.

`self.velocita = 0` stabilisce lo stato iniziale della velocità. Il metodo `accelera` potrà modificarlo in seguito.

La creazione di un oggetto

CODICE PYTHON

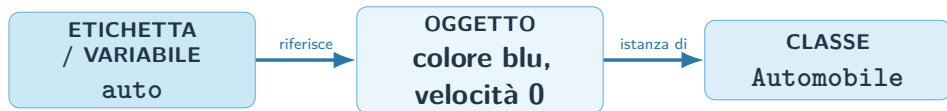
```
# Dopo la definizione della classe  
auto = Automobile("blu")  
print(auto.colore)  
print(auto.velocita)  
print(type(auto) is Automobile)
```

OUTPUT

```
blu  
0  
True
```

1. La chiamata a `Automobile("blu")` crea un'istanza.
2. Python esegue `__init__` su quell'istanza, impostando colore e velocità.
3. L'assegnazione associa l'etichetta `auto` all'oggetto ottenuto.

Classe, oggetto ed etichetta



- ▶ auto è la **variabile**: un'etichetta che fa riferimento all'oggetto.
- ▶ L'**oggetto** è la singola automobile, con identità e stato propri.
- ▶ Automobile è la **classe**: il tipo di quell'oggetto.

“La variabile punta all'oggetto” è una rappresentazione **concettuale**: non implica necessariamente un puntatore a un indirizzo RAM come in C.

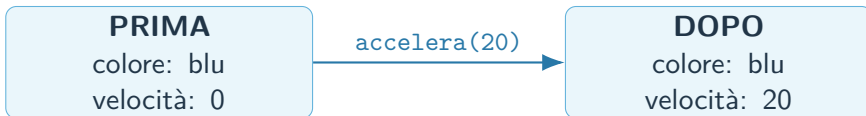
Un metodo modifica lo stato dell'istanza

CODICE PYTHON

```
auto = Automobile("blu")  
print(auto.velocita)  
auto.accelera(20)  
print(auto.velocita)
```

OUTPUT

```
0  
20
```



È lo stesso oggetto in due momenti diversi. Cambia il suo stato; auto continua a fare riferimento a quella stessa istanza.

Due istanze della stessa classe

CODICE PYTHON

```
auto = Automobile("blu")
altra = Automobile("rossa")
altra.accelera(30)
print(auto.colore, auto.velocita)
print(altra.colore, altra.velocita)
print(type(auto) is type(altra))
print(auto is altra)
```

OUTPUT

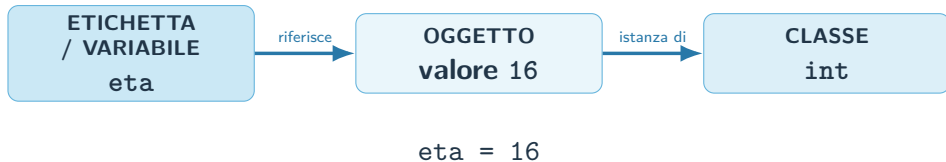
```
blu 0
rossa 30
True
False
```

Il metodo opera sull'istanza riferita da `altra`. La velocità dell'istanza riferita da `auto` rimane 0.

`type(auto) is type(altra)` verifica che la classe sia la stessa. `auto is altra` verifica invece l'identità: qui si tratta di due oggetti distinti.

Le classi già disponibili in Python

Python offre classi pronte all'uso, fra cui `int`, `float`, `bool` e `str`.



- ▶ Il valore 16 è rappresentato da un oggetto della classe `int`.
- ▶ L'etichetta `eta` permette di fare riferimento a quell'oggetto.
- ▶ La classe determina le operazioni disponibili, per esempio l'addizione.

Il modello è lo stesso di `Automobile`. Per usare numeri e stringhe non dobbiamo definire le loro classi: sono già disponibili nel linguaggio.

Un metodo reale: str.upper()

CODICE PYTHON

```
testo = "ciao"  
maiuscolo = testo.upper()  
  
print(testo)  
print(maiuscolo)  
print(type(maiuscolo))
```

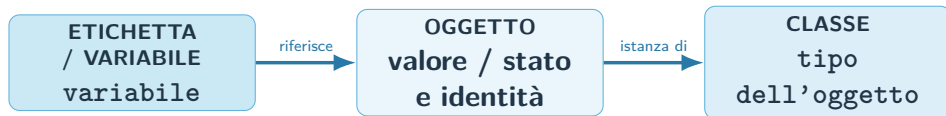
OUTPUT

```
ciao  
CIAO
```

upper() è un metodo della classe str. testo.upper() lo chiama sull'oggetto riferito da testo.

Le stringhe sono **immutabili**. In questo esempio upper() restituisce una stringa con valore "CIAO", riferita da maiuscolo. testo continua a riferire "ciao".

Il modello generale da ricordare



- ▶ La **variabile** è l'etichetta che fa riferimento all'oggetto.
- ▶ L'**oggetto** è un'istanza con una propria identità e un valore o stato.
- ▶ La **classe** ne determina il tipo e i comportamenti disponibili.

La prima freccia significa **fa riferimento a**. La seconda significa **è istanza di**. La classe non è un contenitore nel quale si sposta l'oggetto.

Classe, valore e identità

Consideriamo:

CODICE PYTHON

```
eta = 16

print(type(eta))
print(eta)
print(id(eta))
```

OUTPUT

```
<class 'int'>
16
140...      (esempio)
```

TIPO

int

VALORE

16

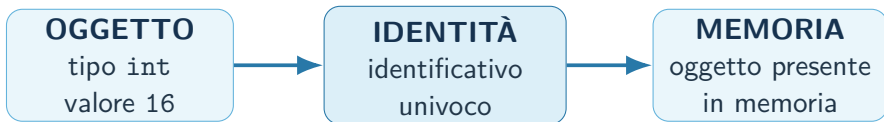
IDENTITÀ

identificativo univoco

- ▶ il **tipo** descrive la natura dell'oggetto;
- ▶ il **valore** è il suo contenuto;
- ▶ l'**identità** permette di distinguere quell'oggetto dagli altri.

Identità dell'oggetto e memoria

L'**identità** identifica univocamente un oggetto durante la sua esistenza.

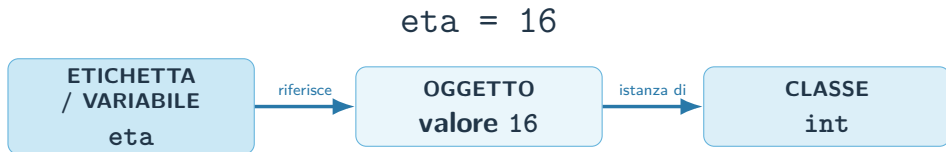


La funzione `id()` restituisce un numero intero che rappresenta l'**identità dell'oggetto**.

- ▶ l'identità rimane invariata durante la vita dell'oggetto;
- ▶ oggetti distinti che esistono nello stesso momento possiedono identità differenti;
- ▶ in **CPython**, `id()` corrisponde all'indirizzo dell'oggetto nella memoria del processo.

Python garantisce il concetto di **identità univoca**; non impone però che ogni implementazione debba rappresentarla mediante un indirizzo di memoria.

La variabile è un'etichetta

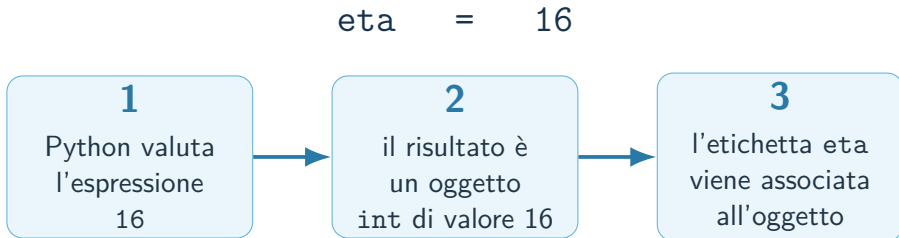


16 è rappresentato da un oggetto della classe `int`. La variabile `eta` fa riferimento a quell'oggetto.

L'etichetta permette di usare l'oggetto nelle istruzioni successive. Non coincide con il valore dell'oggetto né con la sua identità. L'assegnazione associa un'etichetta all'oggetto risultato dell'espressione.

Che cosa significa `eta = 16`?

L'istruzione contiene due elementi:



L'operatore `=` è l'**operatore di assegnazione**: associa l'etichetta presente a sinistra all'oggetto ottenuto valutando l'espressione a destra.

Le classi fondamentali

CODICE PYTHON

```
eta = 16
altezza = 1.75
presente = True
studente = "Anna"
```

```
print(type(eta))
print(type(altezza))
print(type(presente))
print(type(studente))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'str'>
```

Tipo	Significato
int	numero intero
float	virgola mobile
bool	valore logico
str	stringa

Le variabili sono etichette: `type()` restituisce la classe degli oggetti a cui fanno riferimento.

Oggetti della classe int

CODICE PYTHON

```
studenti = 24
temperatura = -3
zero = 0

print(studenti)
print(type(studenti))
```

OUTPUT

```
24
<class 'int'>
```

La classe `int` rappresenta i numeri interi. I suoi oggetti sono immutabili.



Oggetti della classe float

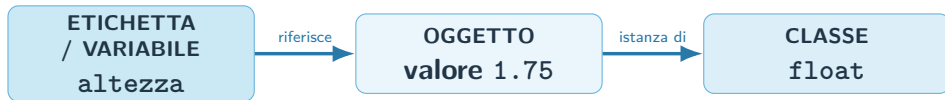
CODICE PYTHON

```
altezza = 1.75  
temperatura = -2.5  
  
print(altezza)  
print(type(altezza))
```

OUTPUT

```
1.75  
<class 'float'>
```

float rappresenta numeri in virgola mobile, con precisione finita. Il separatore decimale è il punto. Normalmente usa 64 bit. Gli oggetti sono immutabili.



Oggetti della classe bool

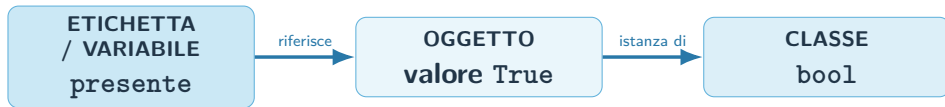
CODICE PYTHON

```
maggiorenne = False  
presente = True  
  
print(maggiorenne)  
print(presente)  
print(type(presente))
```

OUTPUT

```
False  
True  
<class 'bool'>
```

La classe bool ha due valori: True e False. Gli oggetti sono immutabili e si usano per condizioni e confronti.



Oggetti della classe str

CODICE PYTHON

```
studente = "Anna"  
messaggio = "Ciao!"  
numero = "123"  
  
print(studente)  
print(numero)  
print(type(numero))
```

OUTPUT

```
Anna  
123  
<class 'str'>
```

La classe str rappresenta sequenze di caratteri immutabili. "123" è una stringa, mentre 123 è un intero.



Un'etichetta può cambiare oggetto

CODICE PYTHON

```
x = 16

print(x)
print(type(x))

x = "sedici"

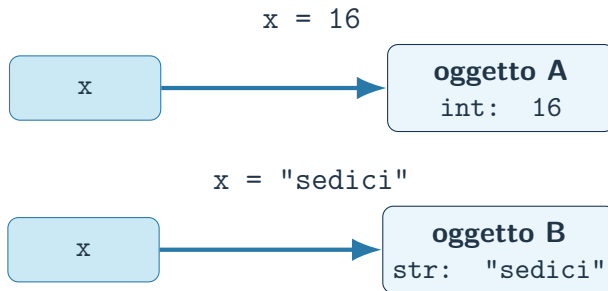
print(x)
print(type(x))
```

OUTPUT

```
16
<class 'int'>
sedici
<class 'str'>
```

L'oggetto intero 16 **non viene trasformato** in una stringa.
La variabile x viene invece associata a un **altro oggetto**.

La riassegnazione cambia il riferimento



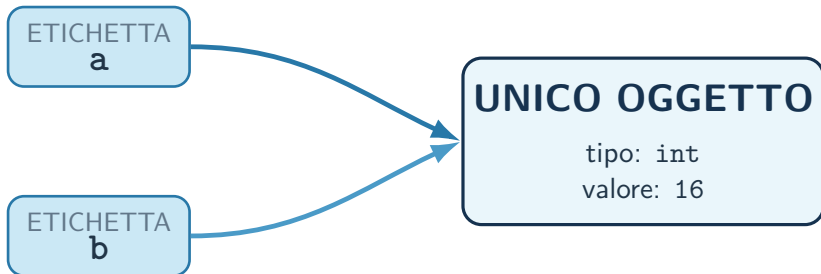
L'etichetta rimane `x`, ma cambia l'**oggetto** a cui fa riferimento.

Aliasing: due etichette, un oggetto

Consideriamo:

`a = 16`

`b = a`



Con `b = a`, Python associa l'etichetta `b` allo **stesso oggetto** a cui fa già riferimento `a`.

2 etichette → **1 oggetto**

Un'operazione produce un oggetto risultato

CODICE PYTHON

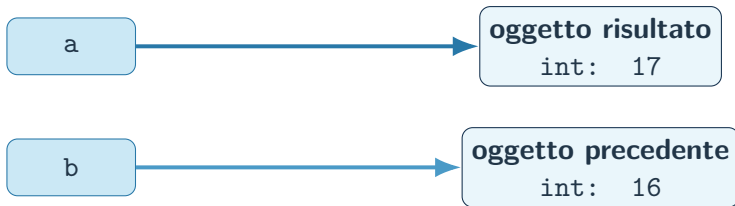
```
a = 16
b = a

risultato = a + 1
a = risultato

print(a)
print(b)
```

OUTPUT

```
17
16
```



Oggetti mutabili e immutabili

La **mutabilità** indica se lo stato di un oggetto può essere modificato dopo la sua creazione.

IMMUTABILI

int
float
bool
str

MUTABILI

list
dict
set
altri oggetti

Riassegnare un'etichetta e **modificare un oggetto** sono operazioni concettualmente differenti.

Aliasing e modifica di un oggetto mutabile

CODICE PYTHON

```
valori = [10, 20]
alias = valori
alias.append(30)

print(valori)
print(alias)
print(valori is alias)
```

OUTPUT

```
[10, 20, 30]
[10, 20, 30]
True
```

Aliasing: più etichette fanno riferimento allo stesso oggetto. `append()` modifica la lista condivisa.

Con `alias = [99]` cambierebbe soltanto il riferimento di `alias`. La variabile `valori` continuerebbe a riferire la lista `[10, 20, 30]`.

Un esempio di oggetto immutabile

CODICE PYTHON

```
parola = "Python"  
  
parola[0] = "J"
```

ERRORE

TypeError

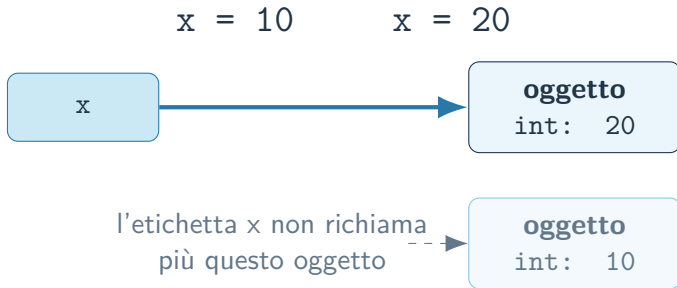
Le stringhe sono **immutabili**.

Non possiamo modificare direttamente un carattere dell'oggetto esistente.

Possiamo invece creare una **nuova stringa**.

Oggetti non più raggiungibili

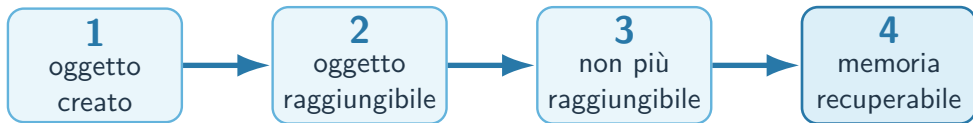
Consideriamo:



Un oggetto diventa **non più raggiungibile** se non esistono altri riferimenti che lo mantengano accessibile. Gli interi piccoli possono essere conservati dall'interprete: lo schema non implica che 10 venga eliminato.

Il garbage collector

Python gestisce automaticamente gran parte della memoria occupata dagli oggetti.



Quando un oggetto non è più raggiungibile, la memoria che occupa può essere **recuperata automaticamente**.

Nell'implementazione **CPython**:

- ▶ il meccanismo principale è il **conteggio dei riferimenti**;
- ▶ è presente anche un **garbage collector** per individuare cicli di oggetti non più raggiungibili.

La funzione print()

CODICE PYTHON

```
studente = "Anna"  
eta = 16  
  
print(studente)  
print("Eta:", eta)
```

OUTPUT

```
Anna  
Eta: 16
```

print() è una **funzione**.

- ▶ riceve uno o più oggetti come argomenti;
- ▶ produce una rappresentazione testuale;
- ▶ la invia allo standard output;
- ▶ restituisce None.

print(): stampare il risultato di un'operazione

CODICE PYTHON

```
a = 4
b = 5
risultato = a * b

print("Primo valore:", a)
print("Secondo valore:", b)
print("Risultato:", risultato)
```

OUTPUT

```
Primo valore: 4
Secondo valore: 5
Risultato: 20
```

L'espressione `a * b` viene valutata **prima della stampa**.

Il risultato dell'operazione è un oggetto e l'etichetta `risultato` viene associata ad esso.

I commenti

CODICE PYTHON

```
# Dati del rettangolo  
base = 5  
altezza = 3  
  
# Creo il risultato  
area = base * altezza  
  
# Mostro il risultato  
print("Area:", area)
```

OUTPUT

Area: 15

Un commento inizia con il simbolo #.

Python ignora il testo successivo sulla stessa riga.

I commenti servono a **documentare il significato del codice**.

La funzione input()

CODICE PYTHON

```
studente = input("Come ti chiami? ")  
  
print("Ciao", studente)  
print(type(studente))
```

ESECUZIONE

```
Come ti chiami? Anna  
Ciao Anna  
<class 'str'>
```

`input()`:

- ▶ può ricevere una stringa da visualizzare;
- ▶ attende l'inserimento dell'utente;
- ▶ restituisce ciò che è stato inserito come oggetto di tipo `str`.

La variabile `studente` è l'etichetta associata alla stringa restituita.

input() restituisce una stringa

CODICE PYTHON

```
eta = input("Eta: ")  
  
print(eta)  
print(type(eta))
```

ESECUZIONE

```
Eta: 16  
16  
<class 'str'>
```

Anche se l'utente digita:

16

la funzione restituisce concettualmente:

"16"

cioè un oggetto di tipo `str`.

Convertire l'input

CODICE PYTHON

```
eta = int(input("Eta: "))
altezza = float(input("Altezza: "))

incremento = 1
eta_futura = eta + incremento

print("Tra un anno:", eta_futura)
print("Altezza:", altezza)
```

ESECUZIONE

```
Eta: 16
Altezza: 1.72
Tra un anno: 17
Altezza: 1.72
```

`input()` restituisce una stringa.

Le chiamate a `int()` e `float()` creano invece un oggetto numerico a partire da una stringa compatibile.

Le principali conversioni

CODICE PYTHON

```
testo_intero = "12"  
testo_reale = "3.5"  
numero = 25  
  
intero = int(testo_intero)  
reale = float(testo_reale)  
testo = str(numero)  
  
print(intero)  
print(reale)  
print(testo)
```

OUTPUT

```
12  
3.5  
25
```

Le classi usate per convertire ricevono un oggetto e, quando la conversione è possibile, **restituiscono un oggetto del tipo richiesto**.

Le classi usate per convertire

Chiamata	Tipo prodotto	Esempio	Risultato
<code>int()</code>	numero intero	<code>int("12")</code>	12
<code>float()</code>	numero in virgola mobile	<code>float("3.5")</code>	3.5
<code>str()</code>	stringa	<code>str(25)</code>	"25"
<code>bool()</code>	valore booleano	<code>bool(1)</code>	True

Una conversione non modifica il tipo dell'oggetto originale.

La chiamata restituisce un oggetto della classe richiesta. In alcuni casi può restituire lo stesso oggetto, per esempio `int(5)`.

Attenzione alla conversione in bool

CODICE PYTHON

```
vuota = ""  
testo_false = "False"  
testo_zero = "0"  
testo = "ciao"  
b1 = bool(vuota)  
b2 = bool(testo_false)  
b3 = bool(testo_zero)  
b4 = bool(testo)  
print(b1)  
print(b2)  
print(b3)  
print(b4)
```

OUTPUT

```
False  
True  
True  
True
```

"" → False

stringa non vuota → True

"False" e "0" contengono caratteri: sono quindi **stringhe non vuote**.

Operazioni di base tra oggetti

CODICE PYTHON

```
a = 7
b = 2

somma = a + b
differenza = a - b
prodotto = a * b
divisione = a / b
div_intera = a // b
resto = a % b
potenza = a ** b

print(somma, differenza, prodotto)
print(divisione, div_intera, resto)
print(potenza)
```

OUTPUT

```
9 5 14
3.5 3 1
49
```

Ogni espressione produce un **oggetto risultato**, che associamo a un'etichetta prima di stamparlo.

In ciascuna print, le virgole separano i valori da mostrare sulla stessa riga.

Gli operatori aritmetici

Operatore	Operazione	Esempio
+	somma	$a + b$
-	sottrazione	$a - b$
*	moltiplicazione	$a * b$
/	divisione reale	a / b
//	divisione intera	$a // b$
%	resto	$a \% b$
**	potenza	$a ** b$

Gli operatori agiscono su uno o più **oggetti operando**.

La valutazione dell'espressione produce un **oggetto risultato**.

Lo stesso operatore con oggetti diversi

CODICE PYTHON

```
a = 10
b = 5
somma = a + b
s1 = "10"
s2 = "5"
unione = s1 + s2
prima = "Py"
seconda = "thon"
parola = prima + seconda
print(somma)
print(unione)
print(parola)
```

OUTPUT

```
15
105
Python
```

```
int + int
→ somma
str + str
→ concatenazione
```

L'operazione dipende dal **tipo degli oggetti coinvolti**.

Quando i tipi non sono compatibili

CODICE PYTHON

```
testo = "Eta: "  
eta = 16  
  
risultato = testo + eta  
  
print(risultato)
```

ERRORE

```
TypeError:  
can only concatenate  
str (not "int") to str
```

Python non definisce l'operazione `str + int`.
Gli oggetti non sono compatibili con quell'operatore.

Rendere compatibili gli oggetti

CODICE PYTHON

```
testo = "Eta: "  
eta = 16  
  
eta_testo = str(eta)  
risultato = testo + eta_testo  
  
print(risultato)  
print(type(eta))  
print(type(eta_testo))
```

OUTPUT

```
Eta: 16  
<class 'int'>  
<class 'str'>
```

L'oggetto 16 rimane un int.

La chiamata a `str()` produce un **oggetto risultato** di tipo `str`.

La precedenza degli operatori

CODICE PYTHON

```
a = 2
b = 3
c = 4

risultato1 = a + b * c
risultato2 = (a + b) * c

base = 2
esponente = 3
fattore = 2
risultato3 = base ** esponente * fattore

print(risultato1)
print(risultato2)
print(risultato3)
```

OUTPUT

```
14
20
16
```

**parentesi → potenze →
moltiplicazioni/divisioni →
somme/sottrazioni**

Esempio di un codice chiaro e leggibile

CODICE PYTHON

```
# Acquisizione
studente = input("Studente: ")
eta = int(input("Eta: "))
altezza = float(input("Altezza: "))

# Elaborazione
incremento = 1
eta_futura = eta + incremento

# Output
print("Studente:", studente)
print("Tra un anno:", eta_futura)
print("Altezza:", altezza)
```

ESECUZIONE

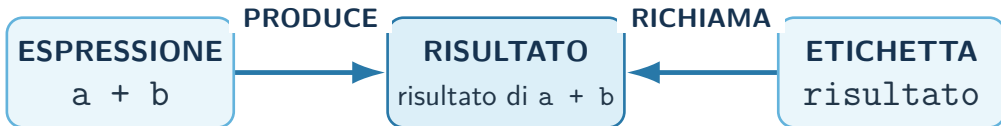
```
Studente: Anna
Eta: 16
Altezza: 1.72
Studente: Anna
Tra un anno: 17
Altezza: 1.72
```

L'elaborazione è separata dall'output: **prima produciamo il risultato, poi lo visualizziamo.**

Il modello mentale da ricordare

Consideriamo l'istruzione:

`risultato = a + b`



- ▶ `a` e `b` fanno riferimento agli oggetti utilizzati come operandi;
- ▶ Python valuta l'espressione `a + b`;
- ▶ la valutazione produce un **oggetto risultato**;
- ▶ l'etichetta `risultato` viene associata a quell'oggetto.

Esercizi — oggetti e tipi

Esercizio 1

Crea quattro oggetti che rappresentino:

- ▶ il testo che identifica uno studente;
- ▶ la sua età;
- ▶ la sua altezza;
- ▶ se è presente oppure assente.

Per ciascun oggetto visualizza:

- ▶ il valore;
- ▶ il tipo mediante `type()`.

Esercizi — conversioni

Esercizio 2

Crea:

```
numero = "25"
```

Trasforma il valore in un oggetto `int` e associa il risultato a una **nuova variabile**.

Disegna l'etichetta che punta all'oggetto.

Visualizza infine:

- ▶ il valore dell'oggetto originale;
- ▶ il suo tipo;
- ▶ il valore del nuovo oggetto;
- ▶ il tipo del nuovo oggetto.

La conversione non modifica l'oggetto originale: restituisce un oggetto della classe richiesta.

Esercizi — input ed elaborazione

Esercizio 3

Realizza un programma che chieda all'utente:

- ▶ come desidera essere chiamato;
- ▶ la propria età.

Crea poi un nuovo oggetto contenente l'età che l'utente avrà tra un anno.

Visualizza:

- ▶ il testo che identifica lo studente;
- ▶ l'età attuale;
- ▶ l'età futura.

L'età ricevuta mediante `input()` deve essere convertita nel tipo corretto **prima di effettuare l'operazione.**

Esercizi — operazioni

Esercizio 4 — Rettangolo

Chiedi all'utente:

- ▶ base;
- ▶ altezza.

Converti entrambi i valori in `float`.

Associa i due oggetti risultato alle variabili:

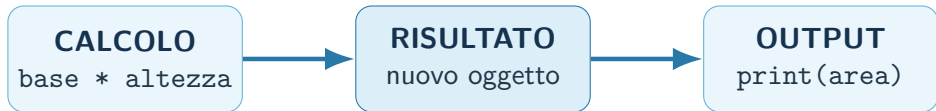
`area` `perimetro`

Le etichette riferiscono rispettivamente il risultato del calcolo dell'area e quello del perimetro.

Infine visualizza i due risultati mediante `print()`.

Esercizi — operazioni

Regola per l'esercizio precedente



Non effettuare il calcolo direttamente dentro `print()`.

Prima crea l'oggetto risultato:

```
area = base * altezza
```

e successivamente visualizzalo:

```
print(area)
```

Esercizi — etichette e riferimenti

Esercizio 5

Senza eseguire il codice, osserva:

```
a = 10
b = a
incremento = 5
risultato = a + incremento
a = risultato
print(a)
print(b)
```

Rispondi:

1. A quale oggetto fa inizialmente riferimento a?
2. Dopo `b = a`, quante etichette fanno riferimento allo stesso oggetto?
3. Quale nuovo oggetto viene prodotto da `a + incremento`?

Esercizi — etichette e riferimenti

Esercizio 5 — continua

Riprendendo il codice della slide precedente:

4. Dopo `a = risultato`, `a` e `b` fanno ancora riferimento allo stesso oggetto?
5. A quale oggetto fa riferimento `a`?
6. A quale oggetto fa riferimento `b`?
7. Quale sarà l'output finale?

Esercizi — individua l'errore

Esercizio 6

Considera il programma:

```
x = "10"  
y = 5  
  
risultato = x + y  
  
print(risultato)
```

Rispondi:

1. Qual è il tipo dell'oggetto riferito da `x`?
2. Qual è il tipo dell'oggetto riferito da `y`?
3. Perché l'operazione `x + y` non è valida?

Esercizi — correggi l'errore

Esercizio 6 — continua

Partendo dal programma della slide precedente:

4. Quale conversione è necessaria se vogliamo ottenere il numero:

15

5. Quale conversione è necessaria se vogliamo ottenere invece la stringa:

"105"

In entrambi i casi crea prima gli oggetti necessari, poi esegui l'operazione e infine stampa il risultato.

Esercizio finale — acquisizione dei dati

Realizza un programma che chieda all'utente:

- ▶ il testo che identifica lo studente;
- ▶ anno di nascita;
- ▶ altezza in metri;
- ▶ numero di ore di studio svolte oggi.

Scegli per ciascun dato il **tipo più appropriato**.

Il programma deve utilizzare:

- ▶ `input()` per acquisire i dati;
- ▶ le conversioni necessarie;
- ▶ variabili con etichette significative;
- ▶ almeno due commenti nel codice.

Esercizio finale — elaborazione

Utilizzando i dati acquisiti nella slide precedente:

1. crea un nuovo oggetto contenente un'età approssimata dello studente;
2. crea gli eventuali altri risultati necessari;
3. mantieni separate le operazioni dall'output;
4. assegna ogni risultato a un'etichetta significativa.

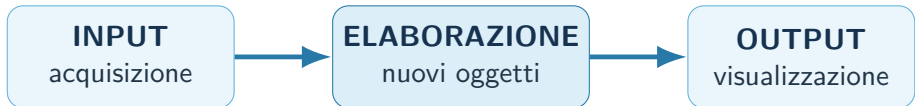
Ogni elaborazione deve seguire il modello:

OGGETTI IN INGRESSO → OPERAZIONE → NUOVO OGGETTO

Esercizio finale — output

Completa il programma visualizzando:

- ▶ tutti i dati acquisiti;
- ▶ l'età approssimata;
- ▶ gli eventuali altri risultati calcolati;
- ▶ il tipo degli oggetti principali mediante `type()`.



Esercizio OOP: classe, istanza e metodo

Usa la classe Automobile presentata nella lezione.

```
auto = Automobile("blu")  
alias = auto  
alias.accelera(15)  
print(auto.velocita)  
print(auto is alias)
```

1. Individua la classe, le etichette e il numero di istanze create.
2. Quale metodo viene chiamato? Quale attributo cambia?
3. Prevedi l'output e disegna le etichette che puntano all'oggetto.
4. Confronta questa modifica con `testo.upper()`: quale oggetto è immutabile?